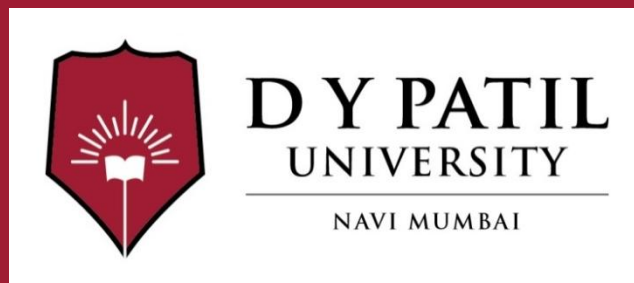


Subject Name : Machine Learning

Unit No: 6

Unit Name : Dimensionality Reduction and Model Deployment

Faculty Name : Dr. Ritu Jain



Unit No.: 6

Faculty Name :

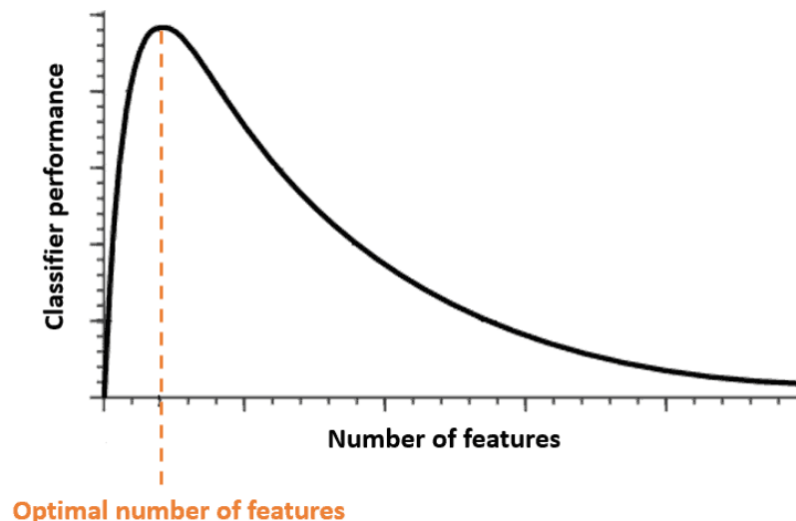
Curse of Dimensionality (CoD)

- The Curse of Dimensionality refers to the various challenges and complications that arise when analyzing and organizing data in high-dimensional spaces (often hundreds or thousands of dimensions).
- In the realm of machine learning, it's crucial to understand this concept because as the number of features or dimensions in a dataset increases, the amount of data we need to generalize accurately grows exponentially.



Curse of Dimensionality (CoD)

- The CoD is closely related to the Hughes Phenomenon. It states that with a fixed number of training instances, the performance of a classifier first increases as the number of features increases until we reach the optimal number of features. Adding more features beyond this value will deteriorate the classifier's performance:



Curse of Dimensionality

- **How does the curse of dimensionality occur?**

- As we add more dimensions to our dataset, the volume of the space increases exponentially. This means that the data becomes sparse.
- Think of it this way:
 - if you have a line (1D), it's easy to fill it with a few points.
 - If you have a square (2D), you need more points to cover the area.
 - Now, imagine a cube (3D) - you'd need even more points to fill the space.
 - This concept extends to higher dimensions, making the data extremely sparse.



What problems does Curse of Dimensionality cause?

- **Data sparsity.** As mentioned, data becomes sparse, meaning that most of the high-dimensional space is empty. This makes clustering and classification tasks challenging.
- **Increased computation.** More dimensions mean more computational resources and time to process the data.
- **Overfitting.** With higher dimensions, models can become overly complex, fitting to the noise rather than the underlying pattern. This reduces the model's ability to generalize to new data.
- **Distances lose meaning.** In high dimensions, the difference in distances between data points tends to become negligible, making measures like **Euclidean distance** less meaningful. In low dimensions, distance measures like **Euclidean distance** can clearly distinguish between **near and far points**. But as the number of dimensions increases:



What problems does Curse of Dimensionality cause?

- **Distances become similar (distance concentration)** The difference between the **nearest neighbor distance** and **farthest neighbor distance** becomes very small.
- **Performance degradation.** Algorithms, especially those relying on distance measurements like **k-nearest neighbors**, can see a drop in performance.
- **Visualization challenges.** High-dimensional data is hard to visualize, making exploratory data analysis more difficult.



How to Solve the Curse of Dimensionality

- In theory, a solution to the CoD could be to enhance the density of data by increasing the number of training instances.
- In practice, it is hard to do. In fact, the number of training observations required to obtain a given density increases exponentially with the number of features.
- **Dimensionality reduction techniques are effective solutions to the CoD.**
- These techniques can be divided into two categories:
 - *feature selection* and
 - *feature extraction.*



 Feature Selection	 Feature Extraction
Selects a subset of relevant features from the original set of features.	Transforms original features into a new, more informative set.
Reduces dimensionality while keeping original features.	Reduces dimensionality by transforming data into a new space.
Methods include Filter, Wrapper and Embedded techniques.	Methods include PCA, LDA, Kernel PCA and Autoencoders.
Requires domain knowledge and feature engineering.	Can be applied to raw data without prior feature engineering.
Enhances interpretability and reduces overfitting.	Improves performance and handles nonlinear relationships.
May lose useful information if important features are removed.	May introduce redundancy and noise if extracted features are not well-defined.

Feature Selection Technique

- Feature selection is a critical preprocessing step in machine learning that reduces data dimensionality by selecting the most relevant features and discarding irrelevant or redundant ones.
- Its primary needs are to increase model accuracy, decrease training time, improve interpretability, and prevent overfitting by eliminating noise.



Need of Feature Selection Techniques

- **Improved Accuracy & Reduced Overfitting:** Removing irrelevant data decreases noise, allowing models to focus on key patterns, which enhances generalization to new data.
- **Reduced Training Time & Computational Cost:** Lowering the number of input features (dimensionality reduction) leads to faster training times and reduced data storage requirements.
- **Increased Model Interpretability:** Models with fewer features are easier for humans to understand and interpret, which is crucial in fields like healthcare or finance.
- **Combating the Curse of Dimensionality:** In high-dimensional datasets, too many features can make it difficult for algorithms to find patterns, leading to suboptimal performance.



Types of Feature Selection Methods

- There are various algorithms used for feature selection and are grouped into three main categories and each one has its own strengths and trade-offs depending on the use case.

1. **Filter Methods**

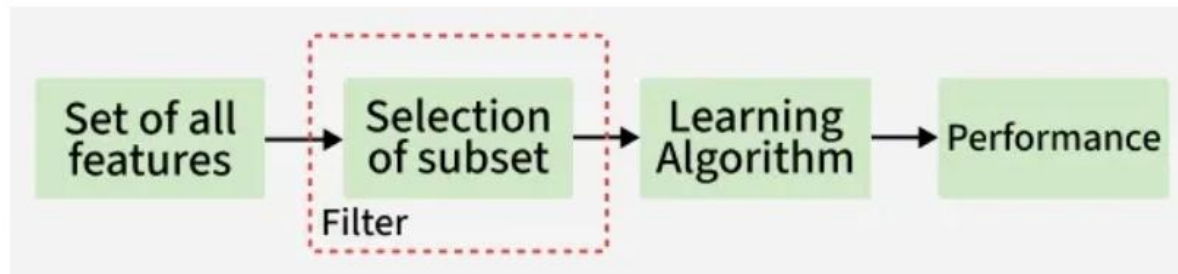
2. **Wrapper methods**

3. **Embedded methods**



Filter Methods

- **Filter Methods are independent of any machine learning algorithm.**
- In the Filter Method, features are selected based on statistics measures. **They use statistical measures to assign a score to each feature based on its relationship with the target variable**
- Filter methods need low computational time and do not overfit the data.
- Filter methods evaluate each feature independently with target variable.
- Feature with high correlation with target variable are selected as it means this feature has some relation and can help us in making predictions.
- These methods are used in the preprocessing phase to remove irrelevant or redundant features based on statistical tests (correlation) or other criteria.



Filter Method



Filter Methods

- The statistical measures used in filter-based feature selection are generally calculated one input variable at a time with the target variable. As such, they are referred to as **univariate statistical measures**. This may mean that any interaction between input variables is not considered in the filtering process.
- **Note:-** In this case, the existence of correlated predictors makes it possible to select important, but redundant, predictors. The obvious consequences of this issue are that too many predictors are chosen and, as a result, collinearity problems arise.



Common Filter Techniques

- **Missing Value:** Feature having the highest proportion of missing value can be dropped.
- **Information Gain:** Calculating the information gain of each variable to the target variable helps to discard useless features.
- **Chi-square Test:** The chi-square value is calculated between each feature and the target variable, and the desired number of features with the best chi-square value is selected.
- **Fisher's Score:** ranks features based on their ability to discriminate between classes, prioritizing features with high between-class variance and low within-class variance Eg: Spam Detection: Identifying words that appear frequently in spam but rarely in non-spam.
- **Variance Threshold:** An unsupervised technique that removes features with very low variance, assuming they contain little information.



Filter methods

- **Advantages of Filter methods**

- **Fast and efficient:** Filter methods are computationally inexpensive, making them ideal for large datasets.
- **Easy to implement:** These methods are often built-in to popular machine learning libraries, requiring minimal coding effort.
- **Model Independence:** Filter methods can be used with any type of machine learning model, making them versatile tools.

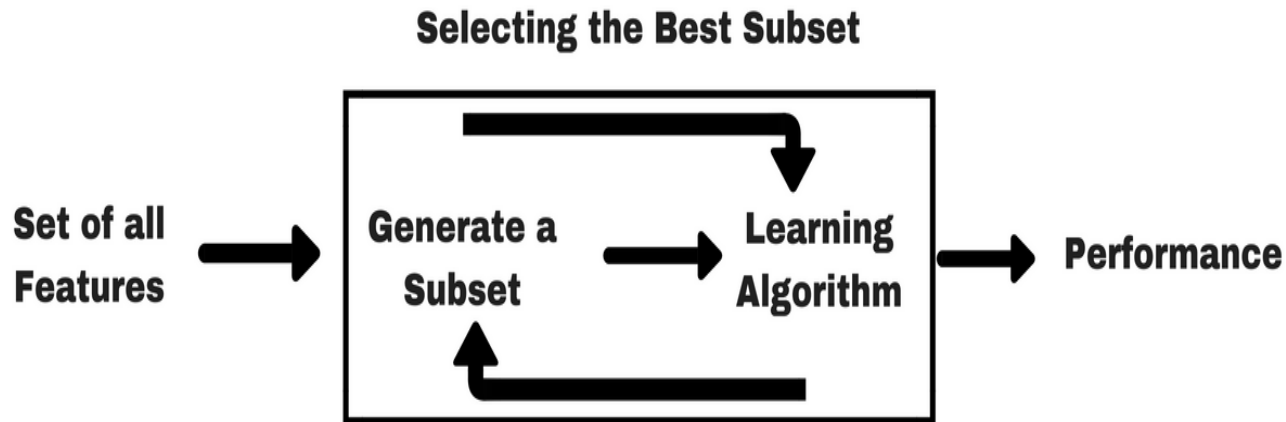
- **Limitations**

- **Limited interaction with the model:** Since they operate independently, filter methods might miss data interactions that could be important for prediction.



Wrapper methods

- Wrapper feature selection methods create many models with different subsets of input features and select those features that result in the best performing model according to a performance metric.
- These methods are unconcerned with the variable types, although they can be computationally expensive.



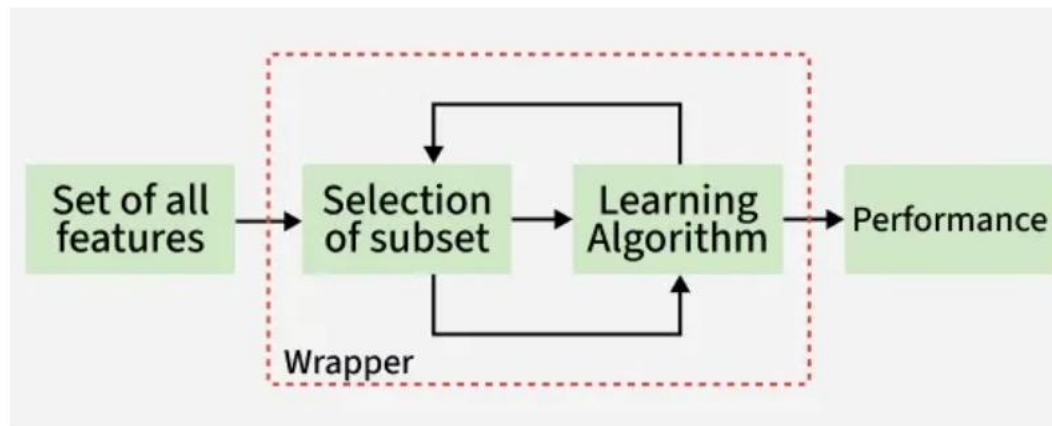
Wrapper methods

- Wrapper methods are called as such because they **"wrap" a machine learning model** around the feature selection process.
- In these techniques, the selection algorithm uses the performance of a specific machine learning model as a criterion to evaluate and select the best subset of features. The model essentially serves as a **"black box"** that provides feedback to the selection procedure
- **Wrapper_methods_**are also referred as greedy algorithms that train algorithm.



Wrapper methods

- They use different combination of features and compute relation between these subset features and target variable and based on conclusion addition and removal of features are done.
- Stopping criteria for selecting the best subset are usually pre-defined by the person training the model such as
 - when the performance of the model decreases or
 - a specific number of features are achieved.



Wrapper Method



Common Wrapper Techniques

- **Forward Selection:** Start with no features and add one at a time based on improvement.
- **Backward Elimination:** Start with all features and remove the least useful ones.
- **Recursive Feature Elimination (RFE):** Removes the least important features step by step.



Common Wrapper Techniques

- **Forward Selection:** Forward selection is an iterative method in which we start with having no feature in the model. In each iteration, we keep adding the feature which best improves our model till an addition of a new variable does not improve the performance of the model.
- **Backward Elimination:** In backward elimination, we start with all the features and removes the least significant feature at each iteration which improves the performance of the model. We repeat this until no improvement is observed on removal of features.
- **Recursive Feature elimination:** It is a greedy optimization algorithm which aims to find the best performing feature subset. It repeatedly creates models and keeps aside the best or the worst performing feature at each iteration. It constructs the next model with the left features until all the features are exhausted. It then ranks the features based on the order of their elimination.



Forward Feature Selection

- Forward feature selection is a greedy iterative algorithm that builds a machine learning model by starting with no features and adding the most significant feature at each step, based on a defined performance metric (e.g., accuracy, precision, etc).
- It improves model interpretability and reduces overfitting by selecting a subset of relevant features.
- Greedy Approach: At each iteration, it evaluates all remaining features and adds the one that offers the best improvement in model performance.

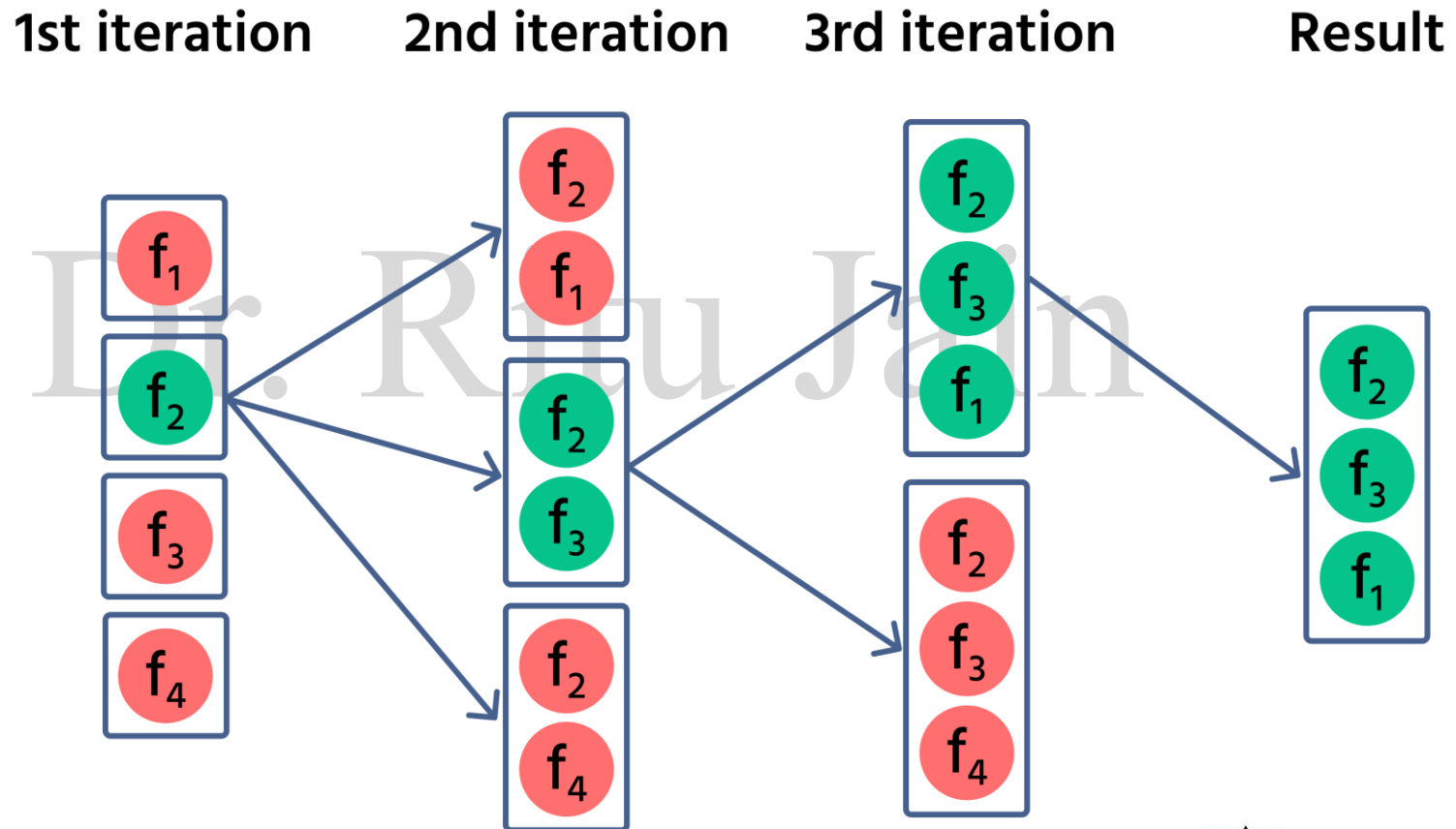


Forward Feature Selection

- This is a bottom-up approach that starts with no features and adds them one by one.
- **Process:** It begins with an empty model and tests each individual feature. The feature that provides the highest improvement in model performance (e.g., accuracy, R-squared) is added.
- **Iteration:** In each subsequent step, it adds the next best feature that, when combined with already selected features, yields the most significant performance boost.
- **Stopping Criteria:** The process stops when no additional features significantly improve the model or a predefined number of features is reached.
- **Best For:** When you have a large number of features and expect only a few to be useful.



Example: Forward Feature Selection



Backward Elimination

- This is a top-down approach that starts with the full set of features and narrows them down.
- **Process:** It begins by training the model with all available independent variables.
- **Iteration:** It identifies and removes the least significant feature (often based on the highest p-value or least impact on performance). The model is then rebuilt with the remaining features.
- **Stopping Criteria:** The process continues until all remaining features are statistically significant (e.g., all p-values are below a threshold like 0.05 or removing further features degrades performance).
- **Best For:** Situations where you want to simplify a complex model by pruning redundant or irrelevant data



Backward Elimination

STEP 1: Select a significance level to stay in the model (e.g. $SL = 0.05$)



STEP 2: Fit the full model with all possible predictors



STEP 3: Consider the predictor with the highest P-value. If $P > SL$, go to STEP 4, otherwise go to FIN



STEP 4: Remove the predictor



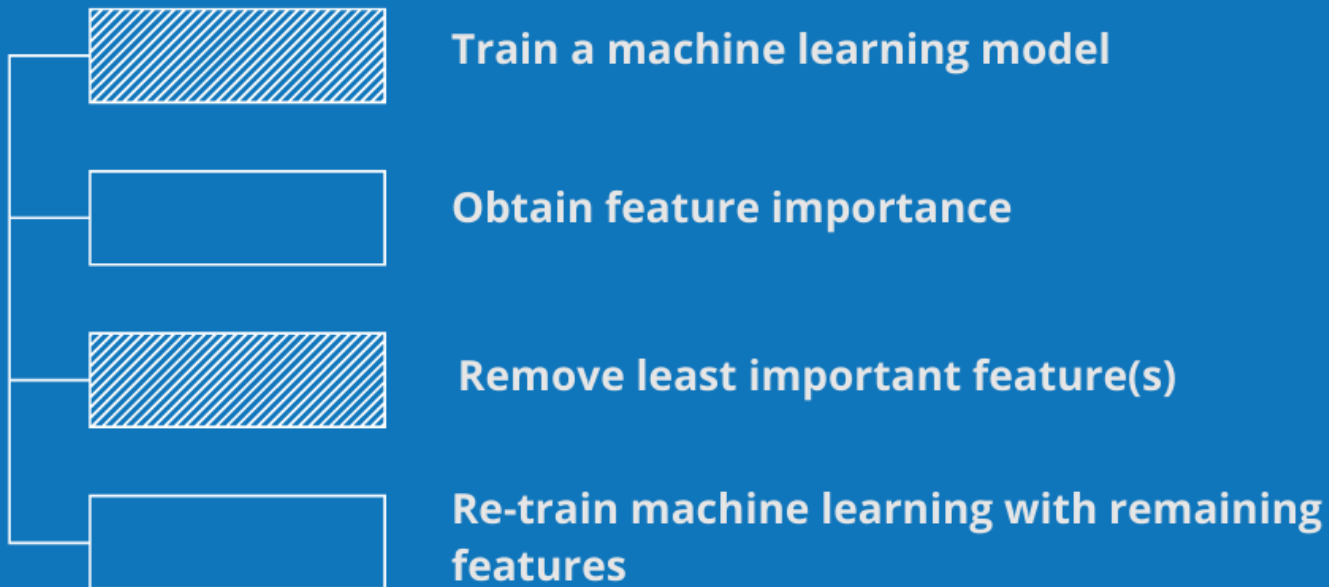
STEP 5: Fit model without this variable*



Recursive Feature Elimination (RFE)

- RFE is a greedy optimization algorithm that recursively removes features to find the best subset.
- **Process:** Similar to backward elimination, it starts with all features, and then the importance of each variable is obtained. If using Scikit-learn, these would be obtained either from the coefficients of a linear regression model (`coef_`) or the importance derived from decision trees (`feature_importances_`). Then, the least important feature or group of features would be removed, and a new machine learning model would be trained using the remaining features.
- **Iteration:** It fits the model, ranks all features by importance, and discards the least important ones. This cycle of retraining and pruning repeats on the smaller feature set.
- **Stopping Criteria:** It continues until the specified number of features is reached.
- **Best For:** High-dimensional datasets where feature interactions are complex, often paired with Support Vector Machines or Decision Trees.

RFE - initial steps



Feature	<u>Forward Selection</u>	<u>Backward Elimination</u>	<u>Recursive Feature Elimination (RFE)</u>
Start	Empty set	All features	All features
Direction	Adding	Removing	Removing
Criterion	Model performance improvement	Statistical significance (p -value)	Internal feature importance
Efficiency	Faster for small feature sets	Better for identifying redundant features	Robust but computationally intensive due to retraining



Advantages and Limitations of Wrapper Methods

- **Advantages**

- **Model-specific optimization:** Wrapper methods directly consider how features influence the model, potentially leading to better performance compared to filter methods.
- **Flexible:** These methods can be adapted to various model types and evaluation metrics.

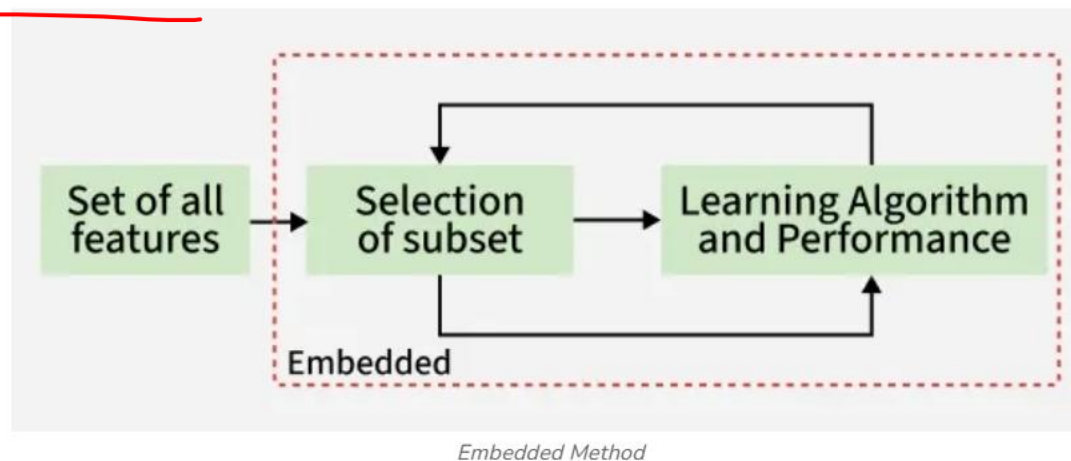
- **Limitations**

- **Computationally expensive:** Evaluating different feature combinations can be time-consuming, especially for large datasets.
- **Risk of overfitting:** Fine-tuning features to a specific model can lead to an overfitted model that performs poorly on unseen data.



Embedded methods

- Embedded methods perform feature selection during the model training process.
- They combine the benefits of both filter and wrapper methods.
- Feature selection is integrated into the model training allowing the model to select the most relevant features based on the training process dynamically.



Common Embedded Techniques

- **L1 Regularization (Lasso)**: Keeps only features with non-zero coefficients.
- **Decision Trees** and **Random Forests**: Select features based on impurity reduction.
- **Gradient Boosting**: Pick features that reduce prediction error the most



Embedded Techniques

- **Advantages**

- **Efficient and effective:** Embedded methods can achieve good results without the computational burden of some wrapper methods.
- **Model-specific learning:** Similar to wrapper methods these techniques use the learning process to identify relevant features.

- **Limitations**

- **Limited interpretability:** Embedded methods can be more challenging to interpret compared to filter methods making it harder to understand why specific features were chosen.
- **Not universally applicable:** Not all machine learning algorithms support embedded feature selection techniques.



Choosing the Right Feature Selection Method

Choice of feature selection method depends on several factors:

- **Dataset size:** Filter methods are generally faster for large datasets while wrapper methods might be suitable for smaller datasets.
- **Model type:** Some models like tree-based models, have built-in feature selection capabilities.
- **Interpretability:** If understanding the rationale behind feature selection is crucial, filter methods might be a better choice.
- **Computational resources:** Wrapper methods can be time-consuming, so consider our available computing power.



Feature Extraction Techniques

- **Feature extraction techniques transform the data from the high-dimensional space to a representation with fewer dimensions.**
- Unlike the feature selection techniques, the feature extraction approach creates new features from functions of the original ones.
- The most commonly used feature extraction techniques are discussed below



Principal Component Analysis (PCA)

- ✓ Principal Component Analysis (PCA) is a linear dimensionality reduction technique that transforms high-dimensional data into a lower-dimensional form while retaining maximum variance.
- ✓ It reduces complexity by creating new uncorrelated variables (principal components), making it ideal for visualizing datasets, improving machine learning efficiency, and reducing data noise.
- ✓ It changes complex datasets by transforming correlated features into a smaller set of uncorrelated components.
- ✓ It helps us to remove redundancy, improve computational efficiency and make data easier to visualize and analyze.



Key Aspects of PCA:

- ✓ **Variance Maximization:** The first principal component (PC1) captures the highest variance, with subsequent components capturing decreasing amounts of variance.
- ✓ **Dimensionality Reduction:** It simplifies complex datasets (e.g., 200 dimensions to 5) while preserving essential information.
- ✓ **Unsupervised Learning:** PCA does not require target labels, making it ideal for exploratory analysis.
- ✓ **Feature Extraction:** Creates new uncorrelated variables (principal components) which are linear combinations of original features, aiding in noise reduction.
- ✓ **Addresses Overfitting:** By reducing the number of input features, PCA helps improve a model's generalization, which minimizes overfitting.
- ✓ **Data Visualization:** Enables visualization of high-dimensional data by reducing it to two or three components for scatter plots.
- ✓ **Common Applications:** Often used for image processing, data compression, and preprocessing before applying machine learning algorithms.



How Principal Component Analysis Works

- PCA uses linear algebra to transform data into new features called principal components.
- It finds these by calculating eigenvectors (directions) and eigenvalues (importance) from the covariance matrix.
- PCA selects the top components with the highest eigenvalues and projects the data onto them to simplify the dataset.
- **Note:** *It prioritizes the directions where the data varies the most because more variation = more useful information.*
- Imagine you're looking at a messy cloud of data points like stars in the sky and want to simplify it. PCA helps you find the "most important angles" to view this cloud so you don't miss the big patterns.



Key Benefits of PCA:

- **Reduced Dimensionality & improved Speed:** By reducing the number of variables, PCA simplifies models and accelerates training time.
- **Noise Reduction:** Low-variance components often represent noise; removing them helps clean the data.
- **Uncorrelated Features:** PCA creates new variables (principal components) that are orthogonal (uncorrelated), handling multicollinearity efficiently.
- **Improved Visualization:** High-dimensional data can be projected down to 2D or 3D for easier visualization and pattern identification.
- **Data Compression:** It requires less storage space for large datasets.



Key Limitations of PCA

- **Interpretability Loss:** Principal components are linear combinations of original features, making them hard to interpret in terms of the original data.
- **Information Loss:** While trying to keep maximum variance, some information is inherently lost.
- **Sensitivity to Scaling:** PCA is sensitive to the variance of features, requiring strict normalization (standardization) before application.
- **Linearity Assumption:** It assumes linear relationships between variables, failing to capture complex non-linear structures.
- **Outlier Susceptibility:** Outliers can significantly skew the covariance matrix and, consequently, the principal components.
- **Computational Cost:** For extremely high-dimensional data, calculating the covariance matrix and its eigenvectors can be slow.



Key Scenarios to Use PCA:

- **High Dimensionality (Curse of Dimensionality):** When your dataset has a very high number of features (columns) compared to observations (rows), causing models to overfit.
- **Highly Correlated Features (Multicollinearity):** When predictor variables are highly correlated, making models unstable. PCA transforms these into uncorrelated principal components.
- **Improving Computational Efficiency:** When training models on high-dimensional data is too slow, reducing variables with PCA speeds up training time.
- **Data Visualization:** When you need to visualize high-dimensional data by reducing it to 2 or 3 components to find clusters or patterns.
- **Data Preprocessing/Denoising:** To remove noise and compress data, focusing on components with the highest variance.
- **When to Use PCA:**
PCA is most effective when dealing with datasets that have many, highly correlated variables, such as in image processing, gene analysis, or high-dimensional financial data.



When NOT to Use PCA

- **Interpretability is Key:** PCA creates new, artificial features that are combinations of original features, making it impossible to explain the influence of specific original variables.
- **Data is Non-Linear:** PCA is a linear technique. If the underlying data structure is highly non-linear, methods like t-SNE or UMAP are better.
- **Small Dataset:** If you have few features, reducing them might eliminate too much useful information.



Key Considerations for PCA

- Feature Scaling: Always scale your data (e.g., using StandardScaler) before applying PCA, as it is sensitive to variance scales.
- Information Loss: PCA reduces dimensions, so some loss of information is unavoidable

Dr. Ritu Jain



Steps in PCA Process

1. **Standardize the Data:** PCA requires scaling data to have zero mean and unit variance.
2. **Compute the Covariance Matrix:** Understand how variables relate.
3. **Compute Eigenvectors and Eigenvalues:** Find eigen vector and eigen value of covariance matrix.
4. **Select Principal Components:** Sort eigen values in descending order and choose top k eigen vectors (principal components) corresponding to k largest eigen values to form a new feature space.
5. **Project Data:** Project the data onto the new reduced feature space



PCA

- In Principal Component Analysis (PCA), all principal components are orthogonal (perpendicular) to each other, meaning they have zero correlation and represent distinct, non-overlapping directions of variance. They are constructed as eigenvectors of the symmetric covariance matrix, which guarantees orthogonality.
- The first principal component (PC1) captures the highest variance. The second (PC2) captures the highest remaining variance under the constraint that it is orthogonal to the first.
- Uncorrelated Components: Because they are orthogonal, the scores of the principal components are uncorrelated.
- Eigenvectors represent the new coordinate system, or principal components, pointing toward the direction of the highest variance in the dataset.
- Eigenvalues indicate how much variance is "stored" along each corresponding eigenvector. A higher eigenvalue corresponds to a more important principal component.



PCA

- **Principal Component Analysis (PCA)** is the most popular dimensionality reduction technique.
- It finds the linear transformations for which the mean squared error computed over a set of instances is minimal.
- The column vectors of the matrix representing this linear transformation are called principal components.
- Original data are projected along with the principal components with the highest variance.
- Hence, the data are transformed to a lower dimensionality that captures most of the information of the original dataset.

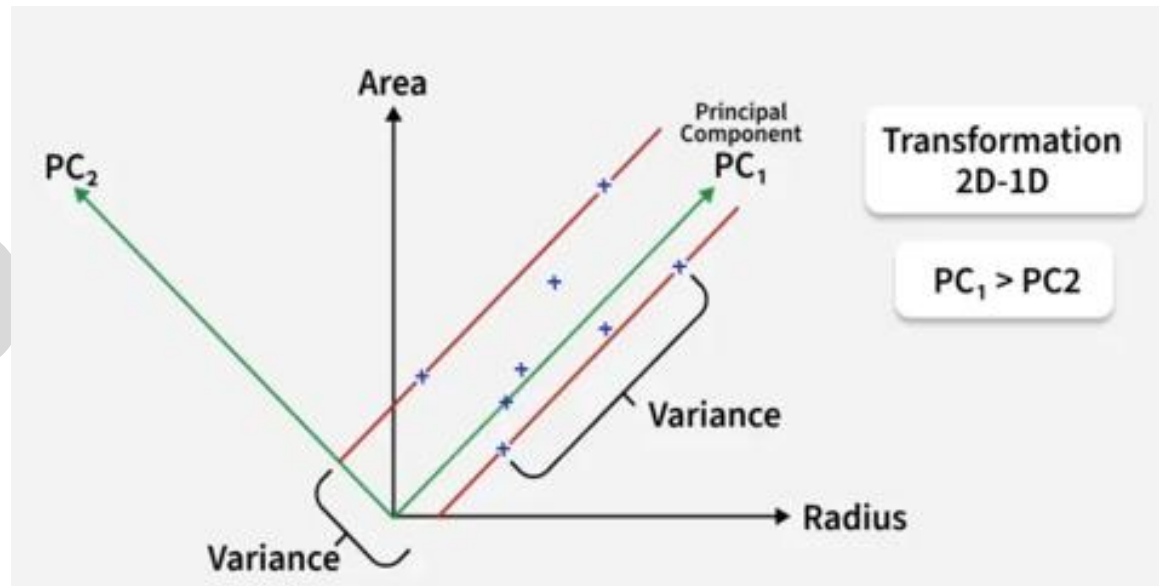


Principal Component Analysis (PCA)

- **PCA** is a statistical method that transforms the original variables into a new set of variables, which are linear combinations of the original variables. These new variables are called principal components.
- Let's say we have a dataset containing information about different aspects of cars, such as horsepower, torque, acceleration, and top speed. We want to reduce the dimensionality of this dataset using PCA.
- Using PCA, we can create a new set of variables called principal components.
- The first principal component would capture the most variance in the data, which could be a combination of horsepower and torque.
- The second principal component might represent acceleration and top speed.
- By reducing the dimensionality of the data using PCA, we can visualize and analyze the dataset more effectively.

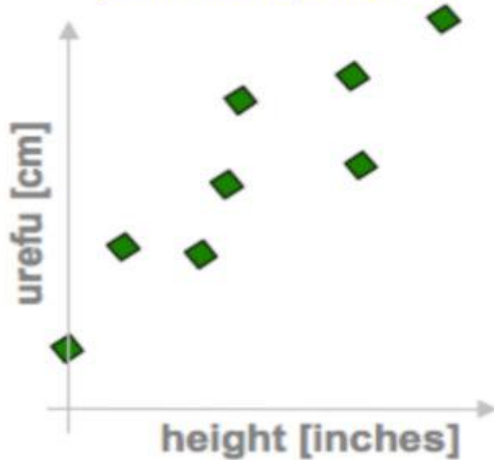


PCA

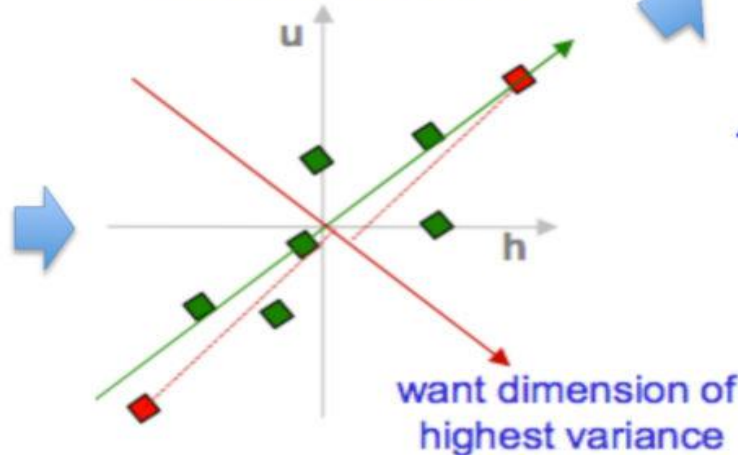


PCA in a nutshell

1. correlated hi-d data
(“urefu” means “height” in Swahili)



2. center the points



3. compute covariance matrix

$$\begin{matrix} & h & u \\ h & \begin{pmatrix} 2.0 & 0.8 \end{pmatrix} \\ u & \begin{pmatrix} 0.8 & 0.6 \end{pmatrix} \end{matrix} \rightarrow \text{cov}(h, u) = \frac{1}{n} \sum_{i=1}^n h_i u_i$$

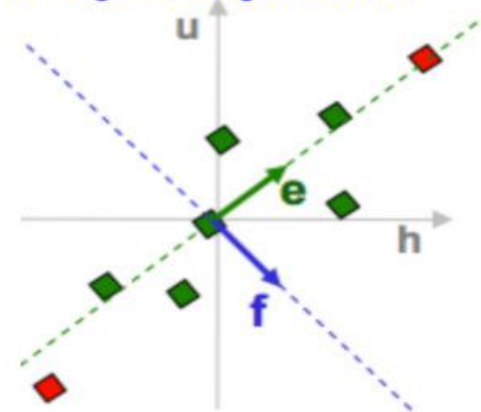
4. eigenvectors + eigenvalues

$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} e_h \\ e_u \end{pmatrix} = \lambda_e \begin{pmatrix} e_h \\ e_u \end{pmatrix}$$

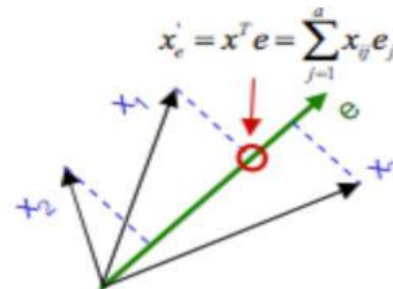
$$\begin{pmatrix} 2.0 & 0.8 \\ 0.8 & 0.6 \end{pmatrix} \begin{pmatrix} f_h \\ f_u \end{pmatrix} = \lambda_f \begin{pmatrix} f_h \\ f_u \end{pmatrix}$$

$\text{eig}(\text{cov}(\text{data}))$

5. pick $m < d$ eigenvectors
w. highest eigenvalues

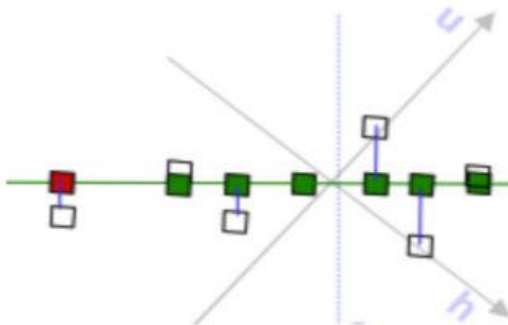


6. project data points to those eigenvectors



Copyright © 2011 Victor Lavrenko

7. uncorrelated low-d data



Linear Discriminant Analysis (LDA)

- **LDA** is a supervised technique used for both classification and dimensionality reduction.
- **Primary Goal**: To project high-dimensional data onto a lower-dimensional space while maximizing the **separability** between categories.
- **Core Principle**: It works by maximizing the distance between class means (**between-class variance**) and minimizing the spread within each class (**within-class variance**).
- This is a classical statistical model used to find a linear combination of features that best separates two or more classes.



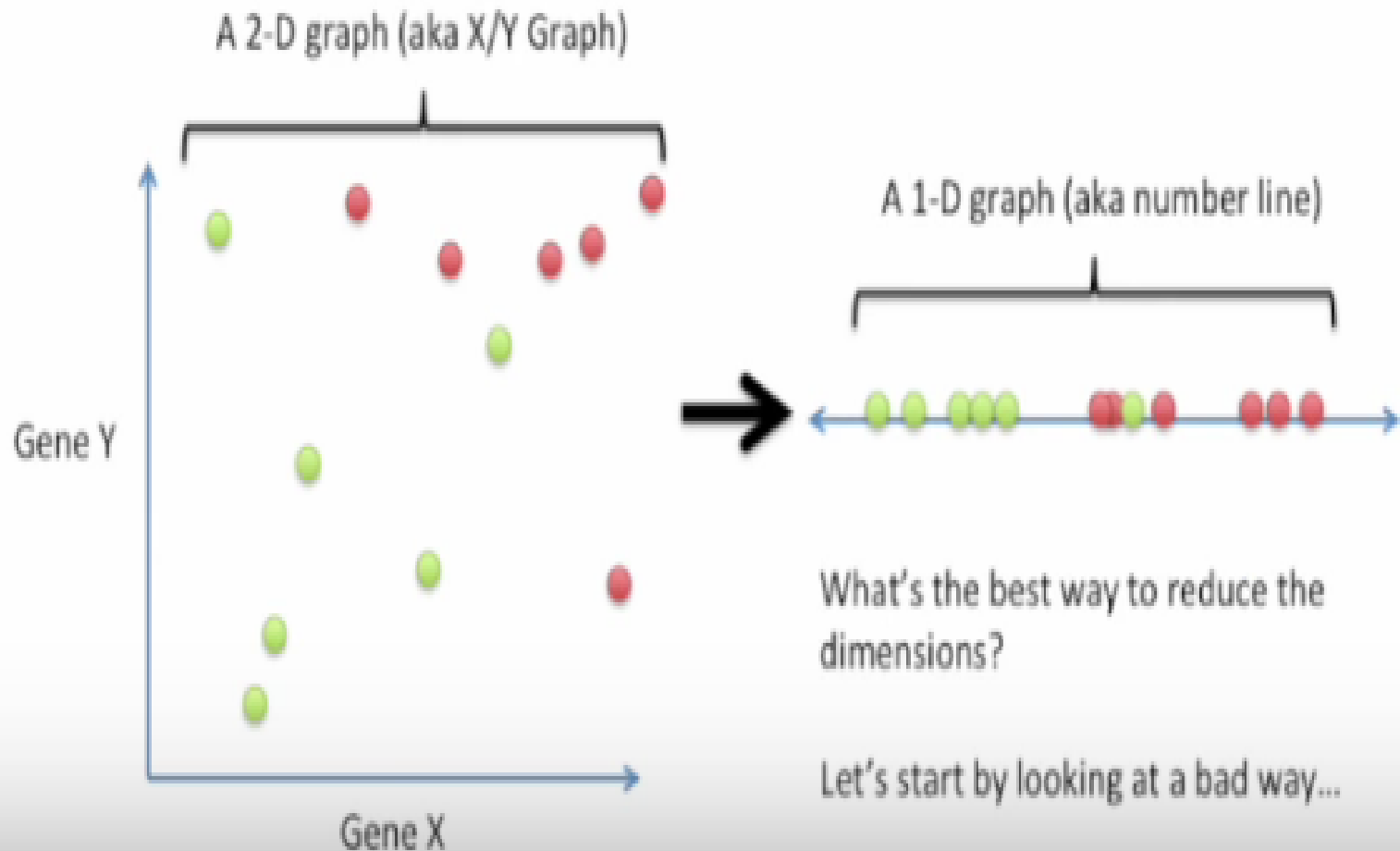
Linear Discriminant Analysis (LDA)

- LDA aims to identify attributes that account for the most variance between classes.
- It's particularly useful for classification tasks.
- **Example:** Suppose we have a dataset with various features of flowers, such as petal length, petal width, sepal length, and sepal width.
- Additionally, **each flower in the dataset is labeled as either a rose or a lily.** We can use LDA to **identify the attributes that account for the most variance between these two classes.**
- **LDA might find that petal length and petal width are the most discriminative attributes between roses and lilies.**
- **It would create a linear combination of these attributes to form a new variable, which can then be used for classification tasks.**
- By reducing the dimensionality using LDA, we can improve the accuracy of flower classification models.



LDA

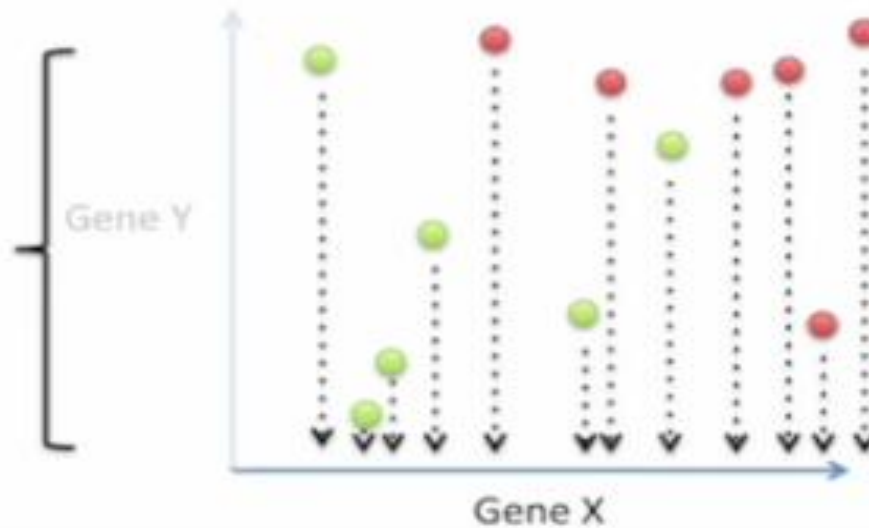
Reducing a 2-D graph to a 1-D graph



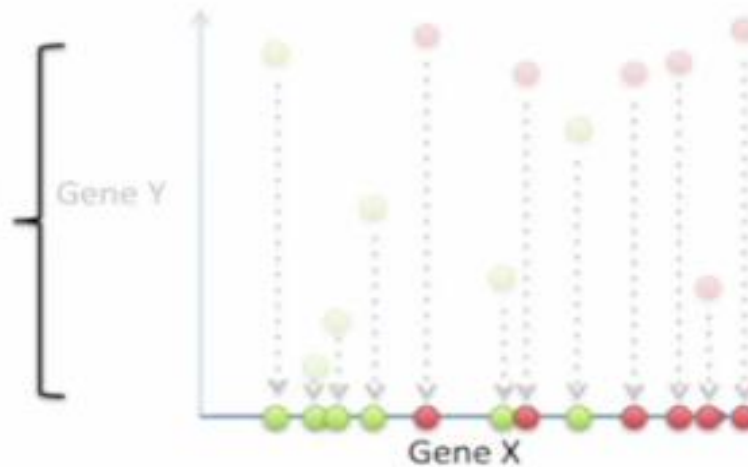
LDA

Bad

One bad option would be to ignore Gene Y



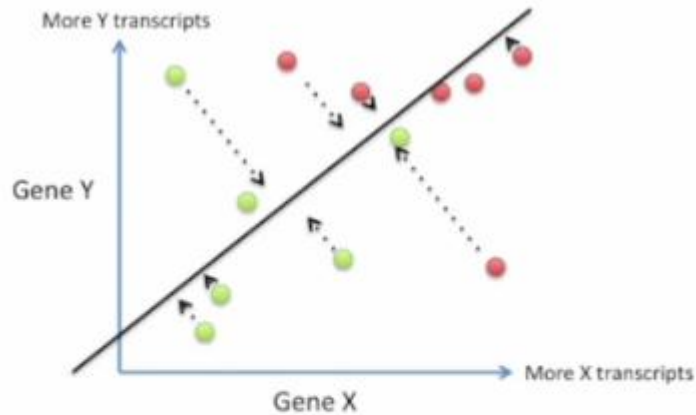
One bad option would be to ignore Gene X



This way is bad because it ignores the useful information that Gene Y provides...

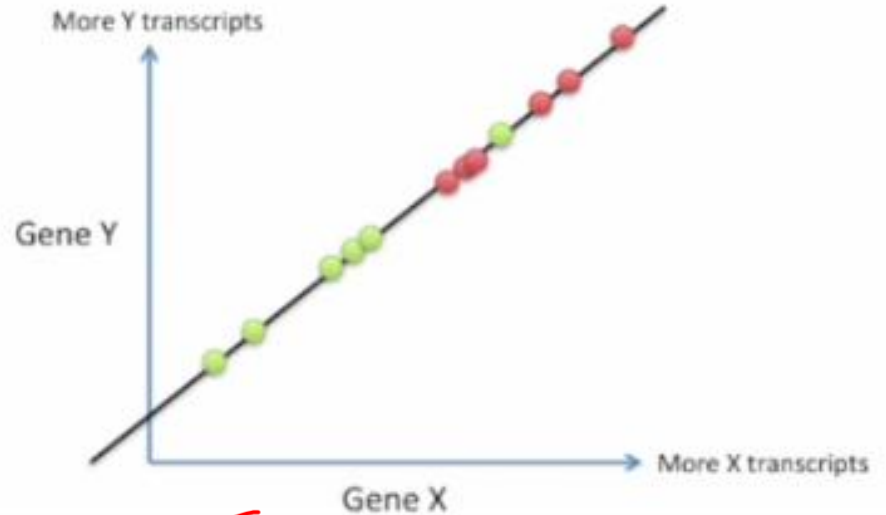
Projecting the genes onto the Y-axis (i.e. ignoring Gene X) isn't any better

LDA



LDA uses both genes to create a new axis...

...and projects the data onto this new axis in a way to maximize the separation of the two categories.



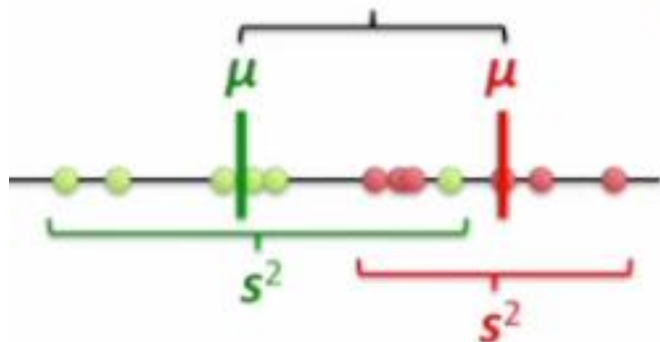
LDA

How LDA creates a new axis...

The new axis is created according to two criteria (considered simultaneously):

1) Maximize the distance between means.

Let's call $(\mu - \mu)$ d for distance.



$$\frac{(\mu - \mu)^2}{s^2 + s^2}$$

Ideally large
Ideally small

2) Minimize the variation (which LDA calls "scatter" and is represented by s^2) within each category.

$$J(W) = \frac{(\mu_1 - \mu_2)^2}{S_1^2 + S_2^2}$$

Where,

$\mu_1 \rightarrow$ Mean of Class 1

$\mu_2 \rightarrow$ Mean of Class 2

$S_1 \rightarrow$ Scatter Matrix of Class 1

$S_2 \rightarrow$ Scatter Matrix of Class 2



LDA

$$J(W) = \frac{(\mu_1 - \mu_2)^2}{S_1^2 + S_2^2}$$

Where,

$\mu_1 \rightarrow$ Mean of Class 1

$\mu_2 \rightarrow$ Mean of Class 2

$S_1 \rightarrow$ Scatter Matrix of Class 1

$S_2 \rightarrow$ Scatter Matrix of Class 2

- **Motivation**

- Find a direction that will amplify the inter-class difference.
- Maximize (squared) difference between the projected means

$$(\mu_1 - \mu_2)^2$$

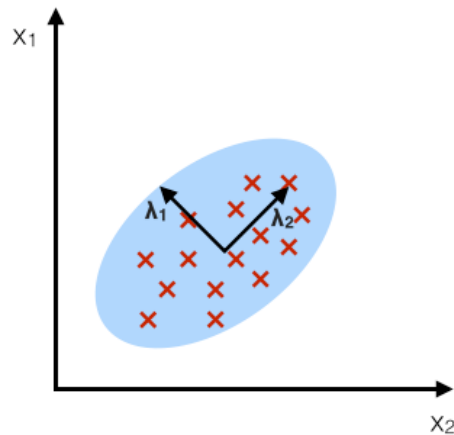
- Minimize the projected scatter within each class

$$S_1^2 + S_2^2$$



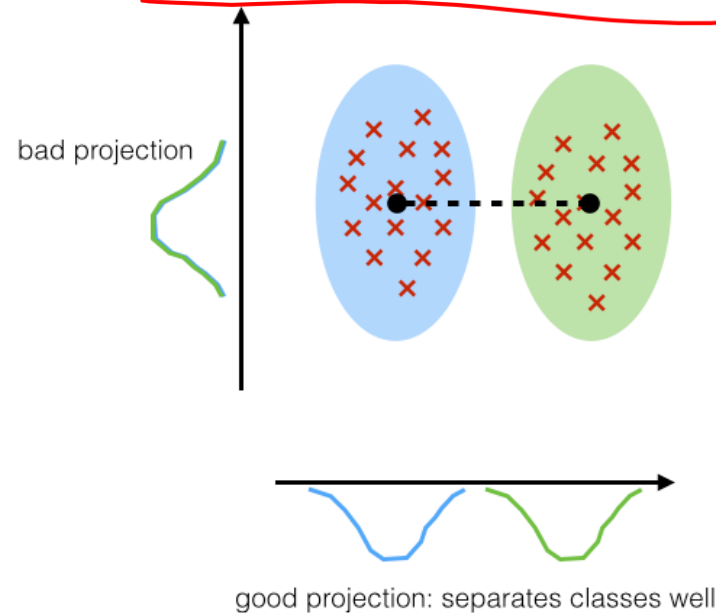
PCA:

component axes that
maximize the variance

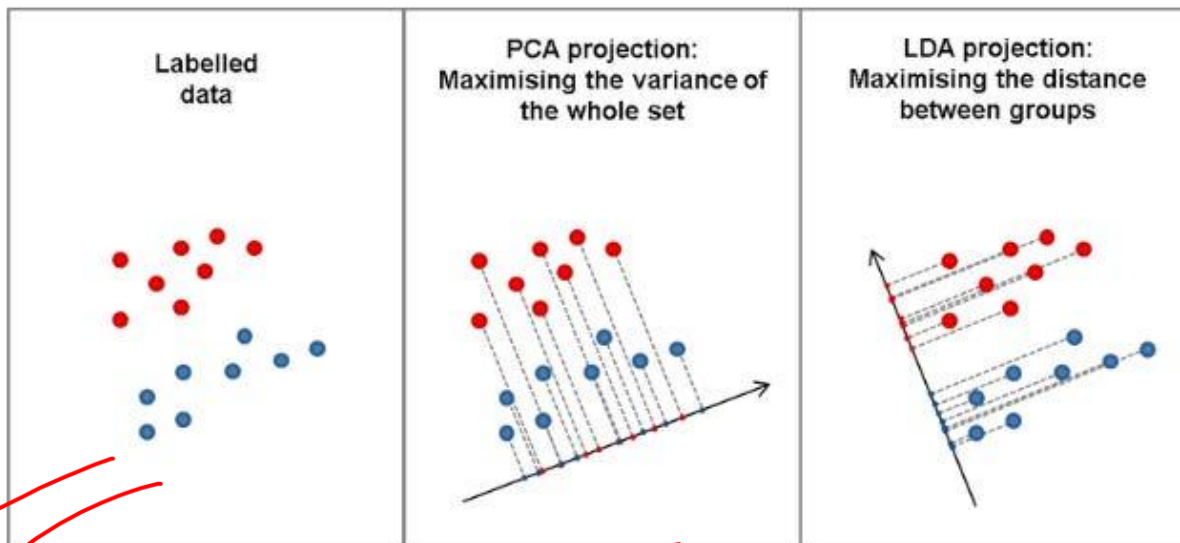


LDA:

maximizing the component
axes for class-separation



Linear Discriminant Analysis



Linear Discriminant Analysis (LDA)

✓ **Comparison with PCA:** Unlike Principal Component Analysis (PCA), which is **unsupervised** and focuses on finding directions of maximum variance regardless of labels, LDA is **supervised** and specifically uses class labels to find the best separation.

• **Common Applications:** Facial recognition, medical diagnosis (classifying disease stages), and customer identification



Key Differences Between PCA and LDA:

- Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) are both linear techniques for dimensionality reduction, but they serve different goals.
- PCA is unsupervised and maximizes data variance, making it ideal for data compression and noise reduction. LDA is supervised, using class labels to maximize class separability, making it better for classification tasks



Key Differences Between PCA and LDA:

- ✓ • **Supervision:** PCA is unsupervised (ignores class labels), while LDA is supervised (requires labels).
- ✓ • **Goal:** PCA finds directions of maximum variance. LDA finds projections that maximize the ratio of between-class variance to within-class variance.
- ✓ • **Best Use Case:** PCA works best when class labels are missing or when preparing data for unsupervised learning. LDA is superior for classification when labelled data is available and classes are well-defined.
- ✓ • **Component Limits:** PCA can produce up to $N-1$ components (where N is the number of features). LDA produces at most $C-1$ components, where C is the number of classes.
- ✓ • **Performance on Small Data:** When the number of samples per class is low, PCA can sometimes outperform LDA.

When to Use Which:

- **Use PCA:** When you want to reduce dimensionality, remove noise, and do not have class labels.
- **Use LDA:** When you are doing classification and want to improve the model performance by focusing on features that differentiate classes.

Dr. Ritu Jain



LDA vs PCA

LDA	PCA
supervised method using known classes (a classifier)	unsupervised
look for linear combinations of features which best explain the data	
maximises the separability between the classes	maximises the variation in the data
rank the new axes based on importance:	
LD1 accounts for the most variation between the classes	PC1 accounts for the most variation in the data



ML system design considerations

- Designing a machine learning (ML) system requires a focus on
 - defining clear business objectives,
 - establishing robust data pipelines, and
 - ensuring scalability, reliability, and maintainability in production.
- Key steps include
 - setting offline/online metrics,
 - performing feature engineering,
 - managing data quality, and
 - implementing automated retraining and monitoring to handle data drift.



ML system design considerations

- Key considerations include:
- ✓ • **Problem Formulation & Metrics:** Clearly define the business goal, identify the ML objective (e.g., classification, ranking), and choose metrics that bridge offline performance with business KPIs.
- ✓ • **Data Engineering & Quality:** Establish data sources, handle data provenance/lineage, ensure data quality, and address labeling requirements.
- ✓ • **Feature Engineering:** Select meaningful features, handle missing data, and ensure consistency between training and serving features (avoiding training-serving skew).



ML system design considerations

- **Model Design & Development:** Start with a simple heuristic model, then move to complex models, using cross-validation and hyperparameter tuning.
- **Deployment & Infrastructure:** Choose between batch or online (real-time) serving, design for scalability (e.g., load balancing, GPU usage), and set up CI/CD pipelines.
- **Monitoring & Maintenance:** Implement automated monitoring for model performance (accuracy, latency) and data drift to trigger automated retraining.
- **Reliability & Safety:** Plan for failures (graceful degradation), ensure fairness, and manage data privacy.

Key tradeoffs, such as performance vs. interpretability and latency vs. accuracy, must be balanced based on the specific use case.



Data Drift, Concept Drift, Data Lineage

- Data drift refers to changes in the **input data's statistical properties** (features), while concept drift refers to changes in the **relationship between input data and the target variable**.
- Data lineage is the process of tracking the lifecycle of data, mapping its journey from origin through transformations to its final destination



Challenges in real-world ML system deployment

1. Data and Management Challenges

- Data is the foundation of any machine learning model, and its quality directly affects the model's performance. The entire ML lifecycle depends on data collection, processing, and management. However, several challenges arise when dealing with data:

a. Data Collection

- Gathering relevant and high-quality data is time-consuming and requires significant effort.
- Data may come from multiple sources, including structured databases, unstructured text, images, and sensor data.
- Privacy and security concerns may restrict data collection, especially in industries like healthcare and finance.
- Data availability may be limited, requiring synthetic data generation or data augmentation techniques.

b. Data Preprocessing

- Raw data often contains missing values, inconsistencies, and noise, requiring extensive preprocessing.
- Feature engineering is necessary to extract meaningful patterns from raw data, which can be labor-intensive and domain-specific.



Challenges in real-world ML system deployment

- **c. Data Augmentation**

- Data augmentation helps improve model generalization by artificially increasing the size of the dataset.
- However, inappropriate augmentation techniques can introduce biases or unrealistic training samples.

- **d. Dataset Size and Storage**

- ML models require large datasets to learn complex patterns effectively.
- Storing and managing large datasets (hundreds of gigabytes or more) can be expensive and computationally intensive.
- Transferring large datasets across systems or cloud environments can lead to delays and high operational costs.



Challenges in real-world ML system deployment

- **e. Data Management and Governance**

- Organizations must implement data governance frameworks to ensure data quality, integrity, and compliance.
- Regulations like GDPR require strict data handling and user privacy protection.

- **2. Model Development and Training Challenges**

- Developing a machine learning model is not just about selecting the right algorithm. Several factors impact model performance and robustness:

- a. Model Selection and Hyperparameter Tuning**

- Choosing the best model architecture requires experimentation with multiple algorithms and configurations.
 - Hyperparameter tuning involves adjusting learning rates, regularization factors, and other settings, which can be computationally expensive.



Challenges in real-world ML system deployment

- **b. Overfitting and Generalization**

- Overfitting occurs when a model learns noise from training data instead of meaningful patterns.
- Ensuring that a model generalizes well to new, unseen data is a major challenge.

- **c. Computational and Resource Constraints**

- Training deep learning models requires significant computational power, often involving GPUs or TPUs.
- Cloud-based training is an option, but it can be expensive and introduce latency in workflows.
- Efficient model training requires optimizing hardware utilization and minimizing redundant computations.



Challenges in real-world ML system deployment

• 3. Deployment and Infrastructure Challenges

- Once a model is trained, deploying it into a production environment presents a new set of challenges.

a. Model Deployment Complexity

- Converting a trained model into an API or a microservice requires additional engineering work.
- Deployment formats vary, with some models needing specialized runtime environments.
- Ensuring compatibility across different cloud platforms and edge devices can be challenging.



Challenges in real-world ML system deployment

- **b. Model Monitoring and Maintenance**

- Deployed models need continuous monitoring to detect performance degradation over time.
- Concept drift and data drift can cause models to become outdated, requiring retraining and fine-tuning.

- **c. Scalability and Latency**

- Real-time applications, such as fraud detection and recommendation systems, require low-latency predictions.
- Scaling ML models to handle high throughput and concurrency involves optimizing inference pipelines.



- **4. Organizational and Strategic Challenges**

- Even with robust technical solutions, organizational and business factors often hinder successful ML deployments.

- a. Lack of ML Expertise**

- Many companies lack experienced ML engineers and data scientists to develop and deploy models effectively.
- Training existing employees and hiring specialized talent can be costly and time-intensive.

- **b. Budget and Resource Allocation**

- ML projects require substantial investment in hardware, cloud services, and personnel.
- Companies often underfund ML initiatives, leading to suboptimal implementations and failed deployments.



Model persistence in Machine Learning

- Model persistence refers to the process of saving a trained machine learning model to a storage medium (like a disk) so it can be reused later without the need for retraining.
- This is essential for deploying models into production where training is a one-time, resource-heavy task, while prediction (inference) must happen quickly and continuously.



Model persistence in Machine Learning

- **Serialization:** Converting a model object into a stream of bytes for storage.
- **Deserialization:** Reconstructing the model object from that byte stream to make new predictions.
- **Pipeline Persistence:** Effective persistence should save the entire prediction pipeline, including preprocessors (scalers, encoders) and metadata, to ensure consistent results.



Common Persistence Methods

- The choice of method depends on your environment and security requirements:
- **Joblib**: The standard for Scikit-learn models. It is optimized for models containing large numerical arrays (like Random Forests) and is faster than standard Python pickle.
- **Pickle**: Python's built-in serialization module. While versatile, it is less efficient for large models and has security risks when loading files from untrusted sources.
- **ONNX (Open Neural Network Exchange)**: An interoperable format that allows you to train a model in one framework (e.g., PyTorch) and run it in another environment (e.g., a C++ application).
- **skops.io**: A more secure alternative to pickle developed specifically for Scikit-learn to avoid executing arbitrary malicious code during loading



Why Persistence Matters

- **Resource Efficiency:** Large models can take days or weeks to train; saving them avoids wasting thousands of dollars in compute costs for every prediction.
- **Workflow Separation:** Data scientists can train models on powerful research machines, while engineers deploy the saved "artifacts" to lightweight production servers.
- **Reproducibility:** Persistence ensures that the model used for auditing or debugging is identical to the one that made the original prediction.
- **Versioning:** It allows teams to track different model iterations, compare their performance, and "hot-swap" or roll back versions in production without service restarts.



Serialization

- Model serialization is the process of converting a machine learning model (or any Python object) into a byte stream that can be stored on disk or transmitted over a network.
- This byte stream can then be deserialized back into the original model, allowing you to reuse the trained model in different environments or at different times.
- Serialization is essential for deploying machine learning models because retraining a model every time you need to use it is impractical and computationally expensive.
- Serializing the model allows you to save the trained model and load it whenever needed.



Pickle: Python's Native Serialization Library

- “Pickling” is the process in which a Python object is converted into a byte stream, and “unpickling” is the inverse operation, whereby a byte stream is converted back into Python objects.
- Pickle is a built-in Python module for serializing and deserializing Python object structures.
- It's simple to use and supports a wide range of Python objects, including machine learning models.
- The code snippet demonstrates how to train a Logistic Regression model using scikit-learn, serialize it to a file named `logistic_regression_model.pkl` using `pickle.dump()`, and then deserialize it back into memory using `pickle.load()`.
- Finally, the loaded model is used to make predictions.
- **Important Note:** Deserializing data from untrusted sources can be dangerous, as Pickle can execute arbitrary code. Use Pickle only with data you trust.



How to Pickle files

- ✓ As usual, you have to start pickling by importing it
 - `import pickle`
- ✓ let's assume that we have a fitted Linear Regression model and we've called it regressor using the following sample:
 - `regressor = LinearRegression()` #Fitting model with training data
 - `regressor.fit(X, y)`
- now we want to save the model to disk. We simple use the `dump()` function in pickle and save the model, as follow:
 - **`pickle.dump(regressor, open('model.pkl','wb'))`**



How to Pickle files (cont'd)

- There are a couple of notes that you must pay attention to.
 - A file name has to be passed in the function. This file name will be used by the code to generate the pickled file. File name does not necessarily require a .pkl extension. It can also be written as `open('model','wb')`
 - When using the `open()` function, 'wb' is required. As explained on top, pickle module uses binary protocol and as a result, the file mode should also be in write and binary format.
- In addition to that, `dump` function is used to write the pickled representation of the object (the model in our example) to the open file.
- **`pickle.dump(obj, file)`**: Write the pickled representation of the object `obj` to the open file object `file`.



De-pickling files

- Let's say that I have my model pickled (or received a pickled file from another data scientist) and need to load it back into my Python program.
- It is very simple and similar to what we saw earlier in pickling process. We need to use the `open()` function again, but this time with 'rb' as second argument. Obviously, r stands for read mode and b is binary, like before.
- The function which is used to de-pickle the file is `load()` from the pickle library. As below:
- # Loading model to compare the results
`model = pickle.load(open('model.pkl','rb'))`



pickle

- As a data scientist, you need to serialise your code (including multiple components) for several reasons such as saving your fitted model on the disk.
- Python has provided the pickle library which makes the life much easier for data scientists who work with ML algorithms all the time.
- Using pickle, simply save your model on disc with `dump()` function and de-pickle it into your python code with `load()` function.
- Use `open()` function to create and/or read from a `.pkl` file and make sure you open the file in the binary format by `wb` for write and `rb` for read mode.



Example of pickle

```
import pickle
from sklearn.linear_model import LogisticRegression
# Train a simple Logistic Regression model
model = LogisticRegression()
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]
model.fit(X, y)
# Serialize the model to a file
filename = 'logistic_regression_model.pkl'
with open(filename, 'wb') as file:
    pickle.dump(model, file)
# Deserialize the model from the file
with open(filename, 'rb') as file:
    loaded_model = pickle.load(file)
# Use the loaded model for prediction
predictions = loaded_model.predict([[0, 0], [1, 1]])
print(predictions)
```



What is Joblib

- ✓ • joblib is a set of tools to provide lightweight pipelining in Python.
- ✓ • It focuses on disk-caching, memoization, and parallel computing.
- ✓ • The library is optimized to be fast and robust on large data in particular and has specific optimizations for NumPy arrays.
- ✓ • Since pipelining is used to efficiently manage a large amount of data, joblib contains a replacement for pickle to work efficiently on large data.



joblib

- **Efficiency:** Optimized for objects with large numerical data, making it faster than standard pickle in data science contexts.
- **Automatic Compression:** Joblib automatically chooses a compression method based on the file extension used.
- **Compression Control:** You can manually set the compression level (0–9) using the compress parameter. A value of 3 is often a good performance-to-size trade-off.



Important Considerations for joblib

- **Security:** Never use `joblib.load` on files from untrusted sources, as it can execute arbitrary code during deserialization.
- **Environment Consistency:** For successful loading, the environment (Python version and library versions like `scikit-learn`) should ideally match the one used during the dump.
- **Custom Classes:** When saving custom classes, ensure the class definition is importable in the environment where you load the file.



Joblib: Optimized Serialization for NumPy Arrays

- Joblib is a Python library that provides optimized serialization and parallelization capabilities, especially for objects that contain large NumPy arrays.
- It's designed to be more efficient than Pickle for serializing and deserializing machine learning models that rely heavily on NumPy, such as scikit-learn models.
- The code snippet demonstrates how to train a RandomForestClassifier model using scikit-learn, serialize it to a file named `random_forest_model.joblib` using `joblib.dump()`, and then deserialize it back into memory using `joblib.load()`. The loaded model is then used to make predictions.
- Joblib uses memory mapping when possible, which can significantly improve performance, especially when dealing with large arrays.



```
import joblib
from sklearn.ensemble import RandomForestClassifier

# Train a RandomForestClassifier model
model = RandomForestClassifier(n_estimators=100)
X = [[0, 0], [0, 1], [1, 0], [1, 1]]
y = [0, 1, 1, 0]
model.fit(X, y)

# Serialize the model to a file using Joblib
filename = 'random_forest_model.joblib'
joblib.dump(model, filename)

# Deserialize the model from the file using Joblib
loaded_model = joblib.load(filename)

# Use the loaded model for prediction
predictions = loaded_model.predict([[0, 0], [1, 1]])
print(predictions)
```



When to use pickle vs joblib

- Joblib is generally preferred for saving machine learning models, especially those built with scikit-learn, because it is significantly faster and more memory-efficient when dealing with large NumPy arrays.
- Pickle, a built-in Python module, is better suited for smaller, general-purpose Python objects and situations where avoiding external dependencies is important. [[1](#), [2](#)]



When to Use Joblib

- ✓ • **Large Data/ML Models:** Use Joblib for models containing large numerical arrays (e.g., scikit-learn models like RandomForest, K-Nearest Neighbors).
- ✓ • **Performance:** It is much faster and more memory-efficient than pickle for large data because it implements specialized serialization for NumPy arrays.
- ✓ • **Compression:** Joblib has built-in, easy-to-use compression (compress=3) to reduce file sizes for large models.
- ✓ • **Memory Mapping:** Joblib can memory-map large objects, allowing them to be loaded without loading the entire object into RAM, which is ideal for huge datasets.



When to Use Pickle

- **Small/Generic Objects:** Ideal for small datasets, dictionary-based objects, or general Python objects that do not heavily rely on NumPy arrays.
- **No External Dependencies:** As part of the Python standard library, pickle does not require installing additional packages, making it better for simple scripts.
- **Maximum Flexibility:** It supports a broader range of complex Python objects, whereas Joblib is specifically optimized for scientific data.



Imp note for Model Persistence

- **Note:** Never unpickle (or unjoblib) data from untrusted sources, as it can execute arbitrary, malicious code on your computer.
- **Alternatives:** For better security and portability of models across different platforms, consider using **ONNX** or **skops.io** instead of either Pickle or Joblib.

Dr. Ritu Jain



What to Use Pickle vs. Joblib to save and load my machine learning models?

- joblib is usually significantly faster on large NumPy arrays because it has special implementations for them. If your model contains large NumPy arrays (as the vast majority of models do), then joblib should be faster than pickle.
- If you don't need to save NumPy arrays, then pickle may be significantly faster, especially on large collections of native Python objects because the pickle module is implemented in C while joblib is pure Python.
- Since recent versions of Python 3.8 (and therefore of the pickle module), it is now much more efficient to pickle large NumPy arrays using the standard library, so the gap is now smaller.
- However, joblib can still be useful to load objects that have nested memory-mapped NumPy arrays. Memory-mapped files are used for accessing small portions of large files on disk, without copying the entire file into memory.



Pickle vs Joblib

- Python's pickle module can serialize almost any object, but it has critical limitations for production ML. It's not version-safe—a model pickled with scikit-learn 1.0 might fail to load in 1.2. It's not secure—loading untrusted pickles can execute arbitrary code. And it's Python-only—you can't load it from Java or Go services.
- joblib is pickle's production-ready cousin. Developed by scikit-learn's team, it compresses models efficiently and handles NumPy arrays better. When Google's search ranking team saves their learning-to-rank models, they use joblib because it's 3-5x faster than pickle for large arrays and maintains backward compatibility across versions.



Feature 	<code>pickle</code>	<code>joblib</code>
Best For	General Python objects (dicts, lists)	Large NumPy arrays & ML models
Optimization	General-purpose, implemented in C	Optimized for large numerical data
Memory	Loads entire object into RAM	Supports memory mapping (<code>mmap</code>)
Compression	Manual setup required	Built-in (zlib, gzip, lz4, etc.)
Installation	Part of Python Standard Library	Requires <code>pip install joblib</code>

. By using memory-mapped files, only parts of the data are loaded into memory as needed, allowing you to process datasets that are larger than your available physical RAM.



Machine learning deployment

- Machine learning deployment is the process of integrating a trained ML model into a production environment, enabling it to deliver real-time or batch predictions from live data.
- This involves packaging the model, hosting it (often as an API), and setting up continuous monitoring, retraining, and versioning to ensure long-term performance.



Streamlit

- **What is Streamlit?**

- **Streamlit** is an open-source Python framework used to build **interactive web apps for data science & ML**
- No frontend (HTML/CSS/JS) required

- Installation

- pip install streamlit

- Run app:

- streamlit run app.py



-
- Simplest program:
 - import streamlit as st

```
st.title("My First App")  
st.write("Hello World")
```

Dr. Ritu Jain



-
- Text Elements:
 - `st.title("Title")`
`st.header("Header")`
`st.subheader("Subheader")`
`st.text("Simple text")`
`st.markdown("**Bold Text**")`
`st.write("Flexible output")`
 - In Streamlit, Markdown is a lightweight markup language used to format and display text without complex code. It serves as the primary way to add **headings**, **bold text**, **lists**, **links**, and **images** to your web applications.
 - **Features Supported**
 - **Text Formatting:** Standard syntax like headers (#), bullet points (*), and blockquotes (>).
 - **Mathematical Equations:** Streamlit supports LaTeX within Markdown blocks for rendering complex math.
 - **Emojis:** You can include [emoji shortcodes](#) (e.g., :sunglasses:) directly in your text.



- User Inputs (Widgets):

- `name = st.text_input("Enter name")`
`age = st.number_input("Enter age", min_value=0, max_value=100)`
`choice = st.selectbox("Select option", ["A", "B", "C"])`
`multi = st.multiselect("Select multiple", ["X", "Y", "Z"])`

Dr. Ritu Jain



- **Buttons:**

```
if st.button("Click Me"):
    st.write("Button Clicked!")
```

Dr. Ritu Jain



What is Flask?

- Flask is a lightweight web application framework.
- It is designed to make getting started quick and easy, with the ability to scale up to complex applications.
- **Flask maps URLs to Python functions and returns responses to the browser.**
- **Used to deploy ML models as web applications / APIs**
- **Helps you connect Model (Python) to Web Interface (User)**



Why Flask for ML?

- Simple and beginner-friendly
- Perfect for **model deployment**
- Works well with **HTML forms / APIs**
- Widely used in **real-world ML apps**

Dr. Ritu Jain



```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def hello_world():
    return "<p>Hello, World!</p>"
```



-
- from flask import Flask

```
app = Flask(__name__)
```

```
@app.route("/")  
def home():  
    return "Hello, Flask!"
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

- How to Run: python app.py



So what did that code do?

- First we imported the **Flask** class.
- Next we create an instance of this class. The first argument is the name of the application's module or package. `__name__` is a convenient shortcut for this that is appropriate for most cases. This is needed so that Flask knows where to look for resources such as templates and static files.
- **App**: Variable storing the Flask application.
- We then use the **route()** decorator to tell Flask what URL should trigger our function. It connects URL → Function
- Meaning: When user visits /, Flask executes the function below it.
- / represents **Homepage**
- Example: `http://127.0.0.1:5000/`



-
- `@app.route("/about")` Opens <http://127.0.0.1:5000/about> and invoke function written just after this decorator
 - The function returns the message we want to display in the user's browser. The default content type is HTML, so HTML in the string will be rendered by the browser.
 - `return "Hello, Flask!"` **Meaning:** Send response back to browser.
 - Flask routes must return something:
 - string
 - HTML page
 - JSON
 - template
 - Save it as `app.py` or something similar. Make sure to not call your application `flask.py` because this would conflict with Flask itself.



-
- `if __name__ == "__main__":` in Python is a conditional statement that ensures certain code blocks only run when a script is executed directly, not when imported as a module. It acts as an entry point, setting `__name__` to `"__main__"` only for the file being executed.
 - This checks whether this file is run directly.
 - **Example**
 - If you run: `python app.py`
 - Then: `__name__` becomes: `"__main__"`
 - So code inside this block executes `app.run(debug=True)`
 - Starts the Flask web server.
 - Flask creates local server: `http://127.0.0.1:5000`
 - You can open this in browser.



-
- **What is debug=True ?**
 - Very important for development.
 - **Benefits**
 - **1. Auto Reload**
 - When you save file: server automatically restarts
 - No need to rerun manually.
 - **2. Error Messages**
 - Shows detailed errors in browser.
 - Very useful while learning.



Full Execution Flow

- **Step 1** Run: `python app.py`
 - **Step 2** Flask server starts.
 - **Step 3** Open browser: `http://127.0.0.1:5000`
 - **Step 4** Browser sends request: `/`
 - **Step 5** Flask finds: `@app.route("/")`
 - **Step 6** Runs: `home()`
 - **Step 7** Returns: `"Hello, Flask!"`
-
- Browser → Flask Route → Function → Response



Example 2: Interpret it

```
from flask import Flask
```

```
app = Flask(__name__)
```

```
@app.route("/")
```

```
def home():
```

```
    return "Home Page"
```

```
@app.route("/about")
```

```
def about():
```

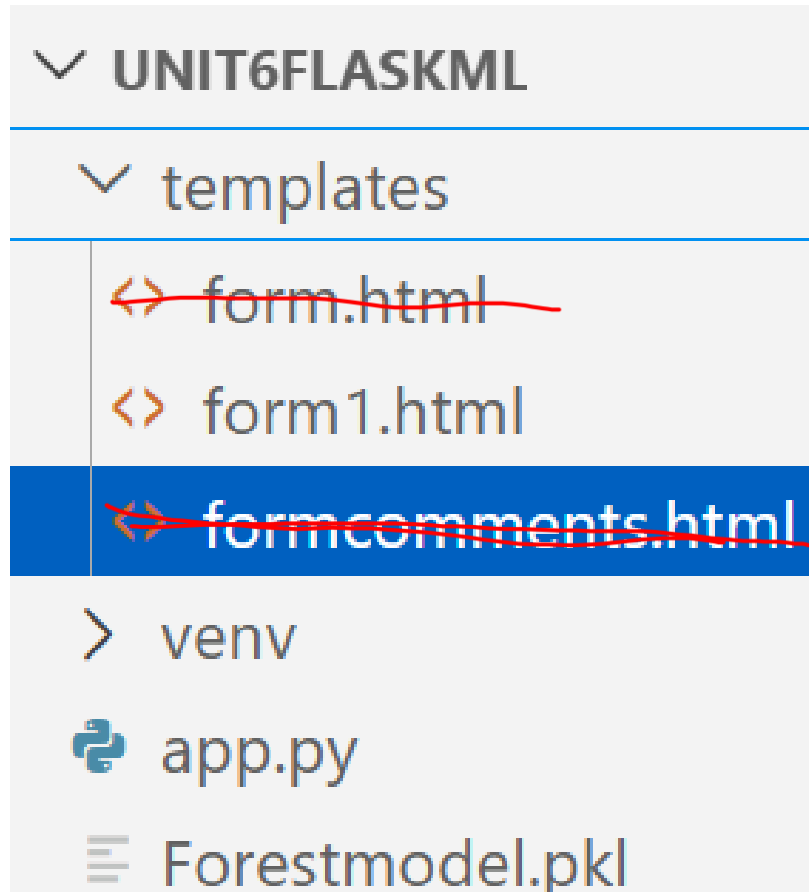
```
    return "About Page"
```

```
if __name__ == "__main__":
```

```
    app.run(debug=True)
```



Flask App Structure



Jain



app.py

app.py > ...

```
1  from flask import Flask, render_template, request
2  import pickle, pandas as pd
3
4  model=pickle.load(open("Forestmodel.pkl","rb"))
5  app=Flask(__name__)
6
7  @app.route("/")
8  def welcome():
9      return render_template("form1.html")
10
11  @app.route("/predict", methods=["POST"])
12  def predict():
13      age=request.form.get('age')
14      salary=request.form.get('salary')
15      result=model.predict([[age,salary]])[0]
16      if result==1:
17          return render_template("form1.html",label=1)
18      else:
19          return render_template("form1.html",label=-1)
20      #return f"The age is {age} and salary is {salary} and prediction is {result}"
21
22  if __name__=="__main__":
23      app.run(debug=True)
```



form.html file

```
<!DOCTYPE html>
<html>
<head>
  <title>Product Purchase Predictor</title>
  <style>
    body{
      font-family: Arial, sans-serif;
      background-color: #f4f4f4;
      text-align: center;
      margin-top: 50px;
    }
    .container{
      width: 300px;
      margin: auto;
      background-color: rgb(186, 251, 252);
      padding: 20px;
      border-radius: 10px;
      box-shadow: 0px 0px 10px gray;
    }
    h2{
      color: #333;
    }
  </style>
</head>
<body>
  <div class="container">
    <h2>Product Purchase Predictor</h2>
    <div class="form">
      <input type="text" value="Name" />
      <input type="text" value="Email" />
      <input type="text" value="Phone" />
      <input type="text" value="Address" />
      <input type="text" value="City" />
      <input type="text" value="State" />
      <input type="text" value="Zip" />
      <input type="button" value="Purchase" />
    </div>
  </div>
</body>
</html>
```



```
input{
  width: 90%;
  padding: 10px;
  margin-top: 8px;
  margin-bottom: 15px;
  border-radius: 5px;
  border: 1px solid gray;
}
.btn{
  background-color: hwb(220 62% 13%);
  color: white;
  border: none;
  cursor: pointer;
  font-size: 16px;
}
.btn:hover{
  background-color: #0056b3;
}
.success{
  color: green;
}
.danger{
  color: red;
}
```

in

</style>

</head>



```
<body>
  <div class="container">

    <h2>Customer Details Form</h2>

    <form action="/predict" method="post">

      <label>Enter Age</label><br>
      <input type="number" name="age" required>

      <label>Enter Salary</label><br>
      <input type="number" name="salary" required>

      <input type="submit" value="Predict" class="btn">

    </form>
```



```
{% if label %}  
    {% if label == 1 %}  
        <h3 class="success">  
            Customer Will Purchase  
        </h3>  
    {% else %}  
        <h3 class="danger">  
            Customer Will Not Purchase the Product  
        </h3>  
    {% endif %}  
{% endif %}  
</div>  
</body>  
</html>
```



<http://127.0.0.1:5000/>

D

Customer Details Form

Enter Age

Enter Salary

Predict



Prediction

Dr.

Customer Details Form

Enter Age

Enter Salary

Predict

Customer Will Purchase the Product



Thank You



D Y PATIL
UNIVERSITY

NAVI MUMBAI